

Capítulo 6. ¿Cómo decide el Sistema Operativo quién utiliza el procesador?

Capítulo 6. Algoritmos de planificación del CPU

Relación con la clase anterior

En la clase anterior estudiamos los procesos, el PCB, el Scheduler, el Dispatcher y el Cambio de Contexto.

Ahora responderemos una nueva pregunta:

¿Cómo decide el Sistema Operativo cuál proceso utilizará el procesador?

Objetivos de aprendizaje

- Comprender por qué es necesaria la planificación del CPU.
- Explicar el papel del Scheduler.
- Analizar los algoritmos FCFS, SJF, Prioridades y Round Robin.
- Comparar ventajas y desventajas de cada algoritmo.

Motivación

Imagina que cinco personas llegan al mismo tiempo a un banco.

Solo existe una ventanilla.

¿Quién debería ser atendido primero?

- El primero que llegó.
- El que terminará más rápido.
- El cliente con mayor prioridad.

- Un turno para cada uno.

Con el procesador ocurre exactamente lo mismo.

¿Por qué es necesaria la planificación?

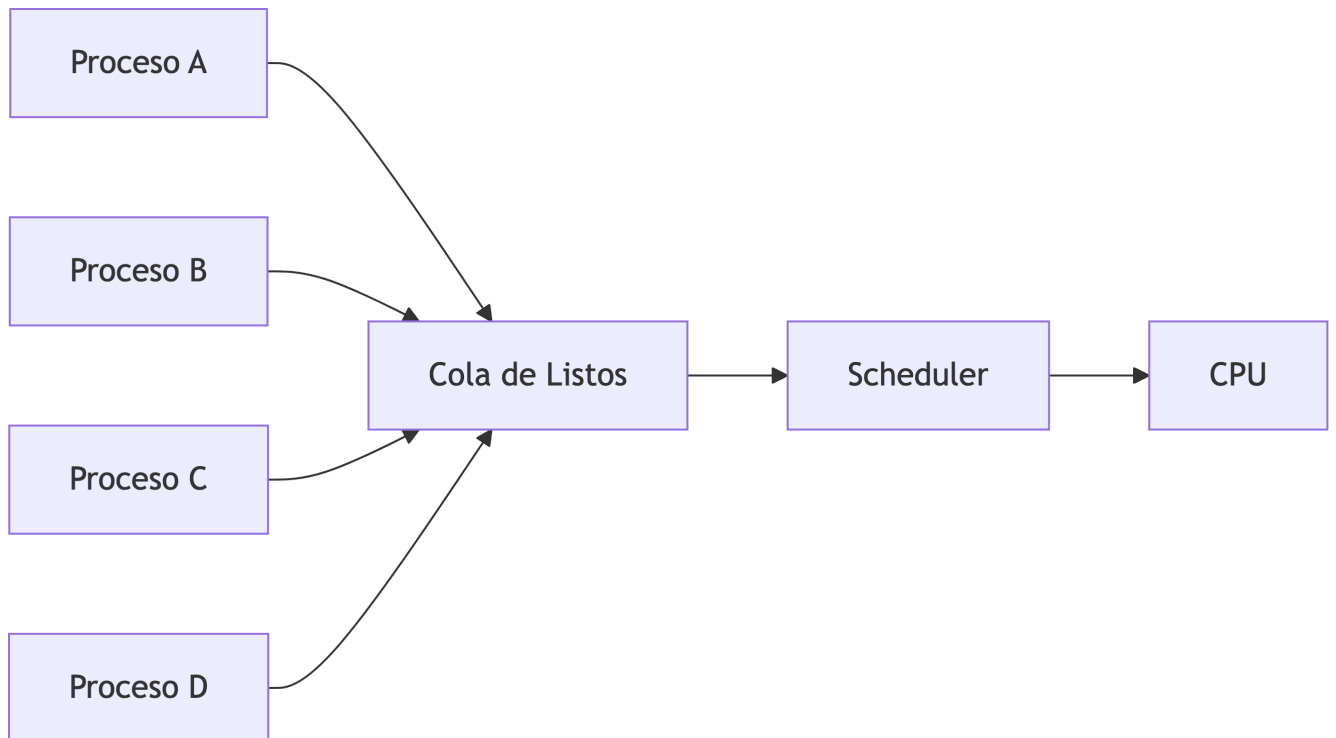
En una computadora pueden existir muchos procesos listos para ejecutarse.

Sin embargo, el CPU únicamente puede ejecutar **un proceso a la vez**.

Por ello el Sistema Operativo necesita decidir:

- ¿Quién utiliza el procesador?
- ¿Durante cuánto tiempo?
- ¿Cuándo debe cambiar a otro proceso?

Ese proceso de decisión recibe el nombre de **Planificación del CPU**.



i Idea clave

La planificación busca utilizar el procesador de la forma más eficiente posible.

¿Qué aprenderemos en este capítulo?

Estudiaremos:

1. FCFS (FIFO)
2. SJF
3. Prioridades
4. Round Robin

Y al finalizar compararemos cuándo conviene utilizar cada uno.

Preparándonos para la siguiente sección

Ahora estudiaremos al componente encargado de tomar estas decisiones: el **Scheduler**, trabajando en conjunto con el Kernel.

El Scheduler, el Dispatcher y el Kernel

Hasta este momento sabemos que existen varios procesos esperando utilizar el procesador.

La siguiente pregunta es:

¿Quién toma la decisión de cuál proceso debe ejecutarse?

La respuesta involucra tres componentes importantes del Sistema Operativo.

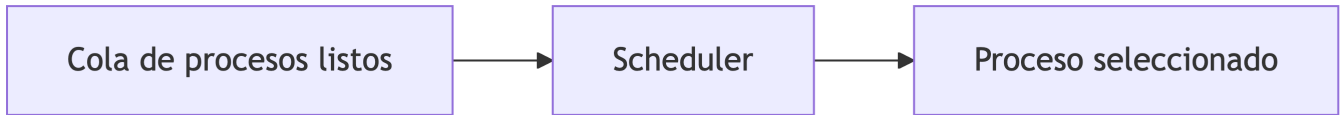
El Scheduler

El **Scheduler** (Planificador) es el encargado de seleccionar el siguiente proceso que utilizará el CPU.

Para tomar esta decisión analiza información almacenada en el PCB de cada proceso.

Entre los principales criterios se encuentran:

- Tiempo de llegada.
- Tiempo estimado de ejecución.
- Prioridad.
- Estado del proceso.
- Algoritmo de planificación utilizado.



i Idea importante

El Scheduler **decide**, pero todavía no entrega el procesador al proceso.

El Dispatcher

Una vez que el Scheduler selecciona el proceso, entra en acción el **Dispatcher**.

Su trabajo consiste en:

- Guardar el contexto del proceso que abandona el CPU.
- Restaurar el contexto del proceso seleccionado.
- Entregar el control del procesador.

Este procedimiento recibe el nombre de **Cambio de Contexto (Context Switch)**.



¿Qué hace realmente el Kernel?

El Kernel coordina todo este proceso.

Mientras el Scheduler decide y el Dispatcher ejecuta el cambio, el Kernel:

- Mantiene la cola de procesos listos.
- Administra los PCB.
- Controla las interrupciones.
- Coordina el cambio de contexto.
- Garantiza que un solo proceso utilice el CPU al mismo tiempo.

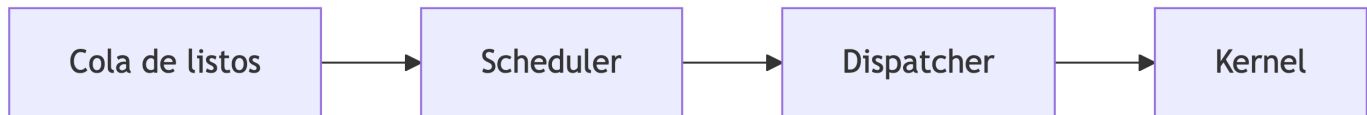
💡 Recuerda

Scheduler → Decide.

Dispatcher → Ejecuta el cambio.

Kernel → Coordina todo el proceso.

¿Cómo trabajan juntos?



Preparándonos para el primer algoritmo

Ahora que comprendemos quién toma las decisiones, comenzaremos con el algoritmo más sencillo de todos:

FCFS (First Come, First Served), también conocido como **FIFO (First In, First Out)**.

Algoritmo FCFS (FIFO)

Ahora estudiaremos el algoritmo de planificación más sencillo.

Su nombre proviene de **First Come, First Served**, que significa:

El primero en llegar, es el primero en ser atendido.

También se conoce como **FIFO (First In, First Out)**.

Un ejemplo cotidiano

Imagina una fila en la cafetería de la universidad.

Los estudiantes llegan en este orden:

1. Ana
2. Luis

3. Carlos
4. María

La regla es muy simple:

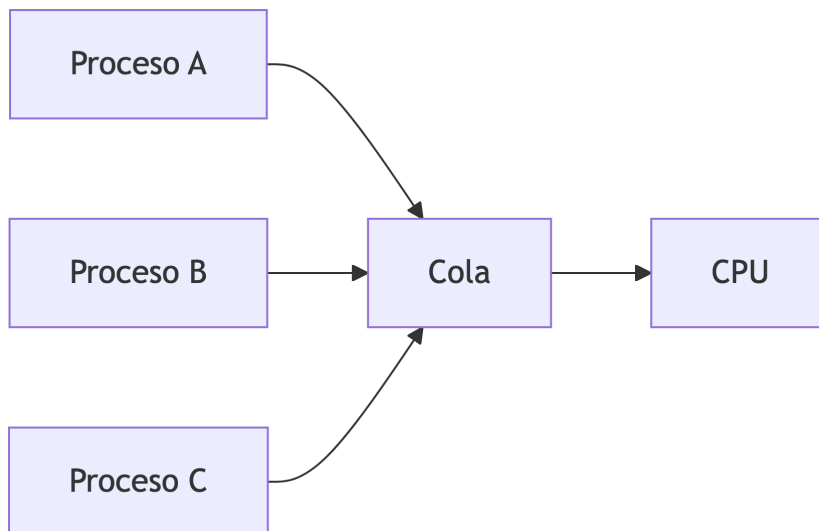
- Nadie puede adelantar a otro.
- El primero en llegar será el primero en ser atendido.

FCFS funciona exactamente igual.

¿Cómo trabaja FCFS?

Cada proceso se coloca al final de la cola de listos.

Cuando el CPU queda libre, el Scheduler selecciona el proceso que lleva más tiempo esperando.



i Idea importante

Una vez que un proceso obtiene el CPU, **no será interrumpido** hasta finalizar su ejecución.

Ejemplo

Utilizaremos los mismos procesos mostrados en la presentación de clase. filecite turn13file0 L5-L8

Proceso	Llegada	CPU
A	0	3
B	1	2
C	3	4
D	4	2
E	7	5
F	9	3

Diagrama de Gantt

```
0      3      5      9      11      16      19
|---A---|--B--|----C----|-D-|-----E-----|-F-|
```

Observa que el orden depende únicamente del momento en que cada proceso llegó al sistema.

Ventajas

- Fácil de implementar.
- Poco costo de administración.
- Ideal para sistemas sencillos.

Desventajas

- Un proceso muy largo puede retrasar a todos los demás.
- Los procesos cortos pueden esperar demasiado.
- No es adecuado para sistemas interactivos.

¿Cuándo utilizar FCFS?

Es recomendable cuando:

- Todos los procesos tienen duraciones similares.
- La simplicidad es más importante que el rendimiento.
- Se trata de sistemas pequeños.

Reflexiona

¿Qué ocurriría si el primer proceso necesitara 30 segundos de CPU y los demás solo 2 segundos?

¿Sería justo utilizar FCFS?

Preparándonos para el siguiente algoritmo

FCFS considera únicamente el orden de llegada.

¿Y si el Sistema Operativo eligiera primero el proceso más corto?

Eso responderá el algoritmo **SJF (Shortest Job First)**.

Algoritmo SJF (Shortest Job First)

Después de conocer FCFS surge una pregunta interesante.

¿Y si en lugar de atender al primero que llegó, atendiéramos primero al proceso que terminará más rápido?

Esa es precisamente la idea del algoritmo **SJF (Shortest Job First)**.

Un ejemplo cotidiano

Imagina una fotocopidora en la universidad.

Hay cuatro estudiantes esperando:

Estudiante	Hojas
Ana	2
Luis	25

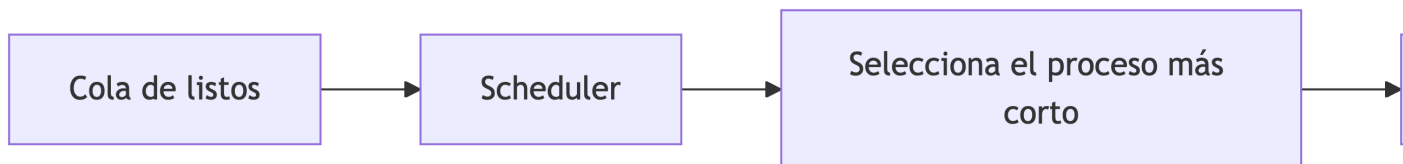
Estudiante	Hojas
Carlos	4
María	1

Si queremos atender al mayor número de personas en el menor tiempo posible, comenzaríamos por los trabajos más cortos.

SJF aplica exactamente esa estrategia.

¿Cómo trabaja SJF?

Cuando el procesador queda libre, el Scheduler selecciona el proceso con el **menor tiempo estimado de CPU** entre los que están listos.



i Idea importante

SJF no observa únicamente el orden de llegada; también considera cuánto tiempo tardará cada proceso en ejecutarse.

Ejemplo

Utilizaremos los mismos procesos del ejercicio de clase. filecite turn13file0 L7-L8

Proceso	Llegada	CPU
A	0	3
B	1	2
C	3	4
D	4	2
E	7	5
F	9	3

Observa el diagrama de Gantt de la presentación y analiza cómo cambian las decisiones del Scheduler al elegir el proceso más corto disponible. filecite turn13file0 L7-L8

Ventajas

- Reduce el tiempo promedio de espera.
- Favorece procesos cortos.
- Mejora el rendimiento en muchas cargas de trabajo.

Desventajas

- Requiere estimar el tiempo de ejecución.
- Los procesos largos pueden esperar demasiado (inanición).

¿Cuándo utilizar SJF?

Es recomendable cuando:

- Se conoce o puede estimarse la duración de los procesos.
- El objetivo es reducir el tiempo promedio de espera.

Error común

SJF no siempre ejecuta el proceso más corto del sistema.
Solo compara los procesos que **ya se encuentran en la cola de listos**.

FCFS vs. SJF

Característica	FCFS	SJF
Criterio	Orden de llegada	Menor tiempo de CPU
Fácil de implementar		
Reduce espera promedio		
Riesgo de espera excesiva (Inanición / Starvation)	Bajo	Alto

i ¿Qué significa el riesgo de inanición (Starvation)?

En Sistemas Operativos, la **inanición** (*Starvation*) ocurre cuando un proceso espera tanto tiempo por el procesador que, en la práctica, casi nunca llega a ejecutarse. Esto sucede porque otros procesos son seleccionados continuamente antes que él.

Ejemplo

Supongamos que tenemos los siguientes procesos:

Proceso	Tiempo de CPU
A	20 s
B	2 s
C	1 s
D	3 s
E	2 s

Si el Sistema Operativo utiliza **SJF (Shortest Job First)**, siempre elegirá primero los procesos con menor tiempo de ejecución.

Si mientras el proceso **A** espera siguen llegando procesos más pequeños, estos continuarán siendo seleccionados antes que él.

Como consecuencia, el proceso **A** podría permanecer esperando durante mucho tiempo. Por esta razón decimos que **SJF presenta un mayor riesgo de inanición (Starvation)** para los procesos largos.

En cambio, con **FCFS**, todos los procesos conservan su turno de llegada, por lo que, tarde o temprano, todos serán ejecutados.

Preparándonos para el siguiente algoritmo

Hasta ahora hemos considerado el orden de llegada y la duración del proceso.

¿Y si algunos procesos fueran más importantes que otros?

Eso nos llevará al algoritmo de **Prioridades**.

Algoritmo de Planificación por Prioridades

Hasta ahora hemos estudiado dos formas de seleccionar el siguiente proceso:

- **FCFS**: el primero que llegó.

- **SJF**: el proceso más corto.

Ahora analizaremos una tercera estrategia.

¿Y si algunos procesos fueran más importantes que otros?

Ese es precisamente el objetivo del algoritmo de **Prioridades**.

Un ejemplo cotidiano

Imagina la sala de emergencias de un hospital.

Llegan cuatro pacientes casi al mismo tiempo.

- Un paciente con un resfriado.
- Una persona con un brazo fracturado.
- Un paciente con un paro cardíaco.
- Una persona con una cortadura leve.

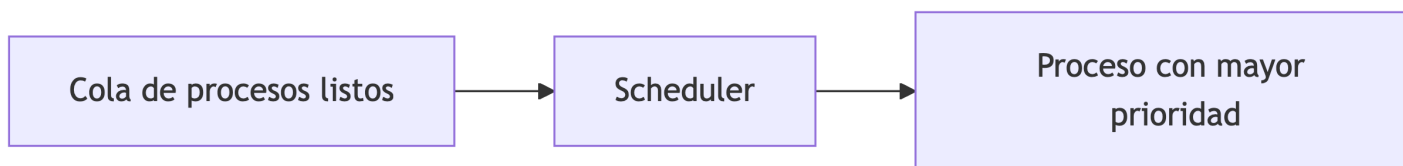
Aunque algunos hayan llegado antes, el médico atenderá primero al paciente cuya condición sea más crítica.

El Sistema Operativo hace algo similar cuando utiliza prioridades.

¿Cómo trabaja?

Cada proceso recibe un **nivel de prioridad**.

Cuando el CPU queda disponible, el Scheduler selecciona el proceso con la prioridad más alta.



i Idea importante

En este algoritmo, **la prioridad es más importante que el orden de llegada**.

Ejemplo

Supongamos los siguientes procesos.

Proceso	Prioridad
A	3
B	1
C	4
D	2

En este ejemplo, **1** representa la prioridad más alta.

El orden de ejecución será:

B → D → A → C

Diagrama de Gantt

```
0       2       5       8       10
|---B---|---D---|---A---|--C---|
```

¿Quién asigna la prioridad?

La prioridad no se asigna al azar.

Generalmente es determinada por el Sistema Operativo considerando aspectos como:

- La importancia del proceso.
- El tipo de aplicación.
- El usuario que ejecuta el programa.
- El tiempo que lleva esperando.
- Las políticas definidas por el Sistema Operativo.

En muchos Sistemas Operativos modernos, la prioridad puede cambiar durante la ejecución para mejorar el rendimiento y evitar que algunos procesos esperen demasiado tiempo.

Ventajas

- Permite atender primero los procesos más importantes.
- Muy utilizado en sistemas operativos modernos.
- Adecuado para sistemas de tiempo real.

Desventajas

- Los procesos de baja prioridad pueden esperar demasiado tiempo.
- También puede presentarse **inanición (Starvation)** si continuamente llegan procesos con mayor prioridad.

⚠ Error común

Muchos estudiantes creen que el proceso con mayor prioridad siempre es el primero que llegó.

Eso es incorrecto.

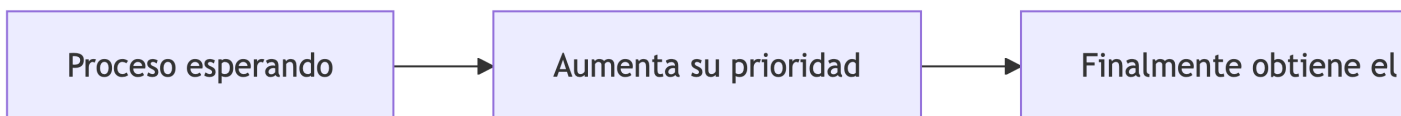
El criterio principal es la **prioridad**, no el tiempo de llegada.

¿Cómo evitar la inanición?

Muchos Sistemas Operativos implementan una técnica llamada **Aging (Envejecimiento)**.

Consiste en aumentar gradualmente la prioridad de los procesos que llevan mucho tiempo esperando.

De esta forma, tarde o temprano obtendrán acceso al procesador.



Comparación

Característica	FCFS	SJF	Prioridades
Criterio principal	Orden de llegada	Menor tiempo de CPU	Mayor prioridad

Característica	FCFS	SJF	Prioridades
Fácil de implementar			
Riesgo de espera excesiva (Starvation)	Bajo	Alto	Alto
Uso recomendado	Sistemas sencillos	Optimizar tiempos	Sistemas críticos

Reflexiona

¿Por qué una alarma de incendio debería ejecutarse antes que un reproductor de música?
 ¿Qué otros procesos de una computadora considerarías de alta prioridad?

Preparándonos para el siguiente algoritmo

Hasta ahora cada proceso conserva el CPU hasta finalizar.

En la siguiente sección estudiaremos **Round Robin**, donde todos los procesos comparten el procesador por turnos mediante un **Quantum**.

Algoritmo Round Robin (RR)

Hasta ahora hemos estudiado algoritmos donde un proceso conserva el procesador hasta terminar su ejecución.

Pero surge un problema.

¿Qué ocurre si un proceso tarda varios minutos y los demás deben esperar?

Para resolver esta situación nació **Round Robin (RR)**.

Un ejemplo cotidiano

Imagina que un profesor debe atender dudas de diez estudiantes.

En lugar de dedicar todo el tiempo a un solo estudiante, decide atenderlos por turnos.

Cada estudiante dispone de **2 minutos**.

Cuando el tiempo termina, pasa el siguiente estudiante.

Si aún tiene dudas, volverá a hacer fila.

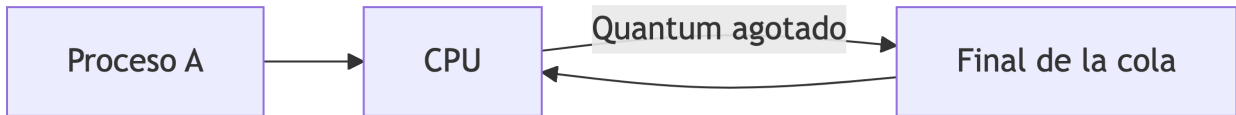
Eso mismo hace Round Robin.

¿Qué es el Quantum?

El **Quantum** es el tiempo máximo que un proceso puede utilizar el CPU antes de cederlo a otro proceso.

Si el proceso termina antes de consumir el Quantum, abandona el procesador normalmente.

Si no termina, el Sistema Operativo lo envía nuevamente al final de la cola de listos.



i Idea clave

Round Robin busca que **todos los procesos tengan oportunidad de utilizar el procesador**, evitando esperas excesivas.

Ejemplo

Supongamos un **Quantum = 2 ms**.

Proceso	Tiempo de CPU
A	5
B	3
C	4

Primera ronda

- A utiliza 2 ms (le quedan 3).
- B utiliza 2 ms (le queda 1).
- C utiliza 2 ms (le quedan 2).

Segunda ronda

- A utiliza 2 ms (le queda 1).
- B utiliza 1 ms y termina.
- C utiliza 2 ms y termina.

Tercera ronda

- A utiliza 1 ms y termina.

Diagrama de Gantt

Quantum = 2 ms

```
0   2   4   6   8   9  11 12
| A | B | C | A | B | C | A |
```

Ventajas

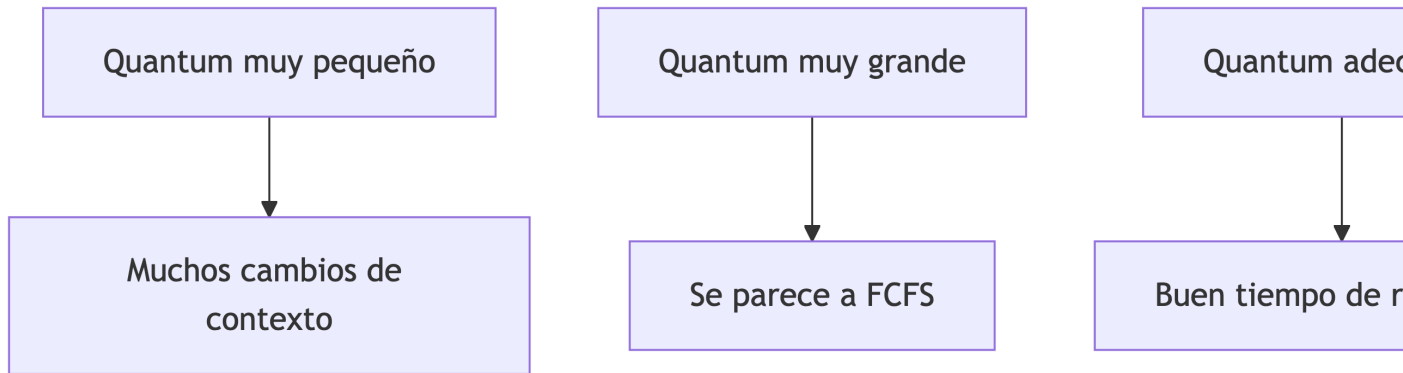
- Todos los procesos reciben tiempo de CPU.
- Muy adecuado para sistemas interactivos.
- Evita que un proceso monopolice el procesador.
- Reduce la sensación de espera del usuario.

Desventajas

- Requiere numerosos cambios de contexto.
- Si el Quantum es demasiado pequeño, aumenta la sobrecarga.
- Si el Quantum es demasiado grande, se parece a FCFS.

¿Cómo elegir un buen Quantum?

- **Muy pequeño:** muchos cambios de contexto y menor rendimiento.
- **Muy grande:** los procesos esperan demasiado.
- **Adecuado:** equilibrio entre rendimiento y respuesta.



Comparación general

Algoritmo	Criterio	Principal ventaja	Principal desventaja
FCFS	Orden de llegada	Muy simple	Esperas largas
SJF	Menor tiempo	Reduce espera promedio	Inanición
Prioridades	Mayor prioridad	Atiende procesos críticos	Inanición
Round Robin	Quantum	Equidad entre procesos	Más cambios de contexto

💡 Reflexiona

¿Por qué Windows, Linux y Android utilizan algoritmos basados en Round Robin para atender aplicaciones interactivas?

¿Qué ocurriría si un navegador web tuviera que esperar varios segundos para volver a recibir tiempo de CPU?

Preparándonos para el cierre

Ya conocemos los cuatro algoritmos clásicos de planificación.

En la siguiente sección los compararemos mediante un caso integrador y resumiremos cuándo conviene utilizar cada uno.

Comparación general de los algoritmos de planificación

Llegamos al final del capítulo.

Ahora ya conocemos los cuatro algoritmos clásicos de planificación del CPU.

Antes de continuar con el siguiente tema, comparemos sus principales características.

Algoritmo	¿Cómo decide?	Ventaja principal	Desventaja principal	Uso recomendado
FCFS	Primer proceso en llegar	Muy sencillo de implementar	Puede generar largas esperas	Sistemas simples o por lotes
SJF	Menor tiempo de CPU	Reduce el tiempo promedio de espera	Riesgo de inanición	Procesos con duración conocida
Prioridades	Mayor prioridad	Atiende primero procesos críticos	Riesgo de inanición	Sistemas de tiempo real
Round Robin	Turnos mediante Quantum	Todos reciben tiempo de CPU	Mayor cantidad de cambios de contexto	Sistemas interactivos

Caso integrador

Supongamos que llegan simultáneamente los siguientes procesos:

Proceso	CPU	Prioridad
A	8	3
B	2	1
C	4	4
D	3	2

Analiza

1. ¿Qué proceso ejecutaría FCFS primero?
2. ¿Qué proceso elegiría SJF?
3. ¿Qué proceso seleccionaría Prioridades?

4. Si el Quantum = 2 ms, ¿cómo comenzaría Round Robin?

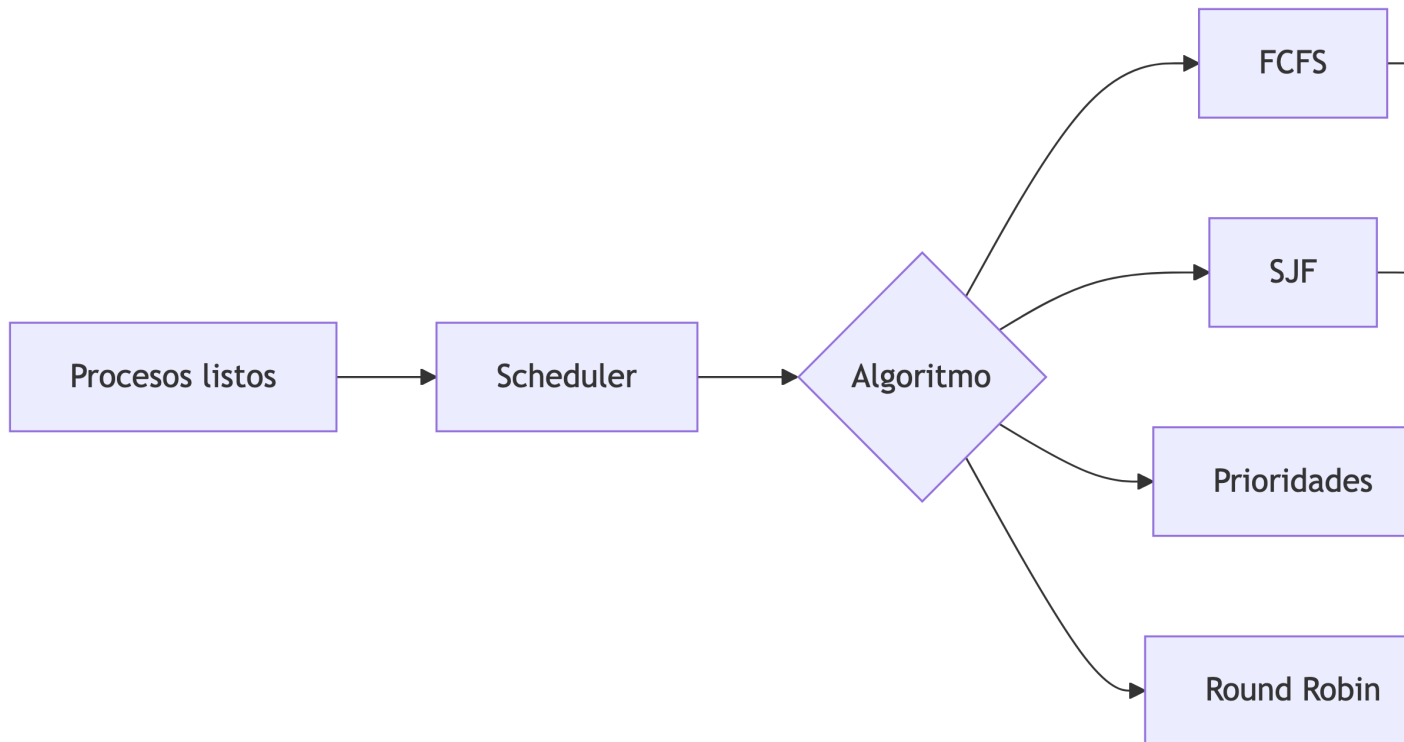
Discute y justifica tus respuestas.

Autoevaluación

1. ¿Cuál es el objetivo de la planificación del CPU?
 2. ¿Qué componente selecciona el siguiente proceso?
 3. ¿Qué función cumple el Dispatcher?
 4. ¿Qué es el Quantum?
 5. ¿Cuál algoritmo utiliza el orden de llegada?
 6. ¿Cuál selecciona el proceso más corto?
 7. ¿Cuál atiende primero procesos críticos?
 8. ¿Qué significa inanición (Starvation)?
 9. ¿Qué técnica ayuda a evitarla?
 10. ¿Por qué Round Robin es adecuado para sistemas interactivos?
-

Resumen del capítulo

Durante esta clase aprendimos que el Sistema Operativo utiliza diferentes algoritmos para decidir qué proceso obtiene el procesador.



💡 Idea clave

No existe un algoritmo perfecto.
Cada Sistema Operativo selecciona el algoritmo que mejor se adapta a sus necesidades.

Conexión con el siguiente capítulo

En el próximo capítulo estudiaremos las **capas del Sistema Operativo** y comprenderemos por qué el **Kernel** es considerado el corazón del sistema.